FinMarket-Enterprise

Phase II — Database and Security Integration

Member 5: @Godwin-pixel — Automation and Analytics

Branch: `feature/automation-p2-m5`
Assigned module: `backend/src/modules/automation/`
Primary ownership: Automation rule persistence, activity event tracking, event processing, trigger execution, scheduled follow-ups, duplicate execution prevention, analytics, and automation authorization.

---

1. Project Overview

FinMarket-Enterprise is a financial advisory marketing and client acquisition platform designed to help financial advisors manage prospective clients, organize marketing campaigns, deliver authorized marketing communications, automate follow-up activities, and monitor marketing performance.

The application consists of five main modules:

1. Identity and Advisor Management
2. Lead Management (CRM)
3. Marketing Campaigns and Templates
4. Email and Facebook Delivery
5. Automation and Analytics

During Phase I, the Automation and Analytics module was implemented using temporary in-memory storage. Core API endpoints were established for creating automation rules, retrieving rules, simulating business events, retrieving event history, and generating analytics summaries.

Initial Mongoose models, automation triggers, supported actions, simulated event processing, and duplicate event validation were also introduced.

Phase II focuses on replacing temporary automation storage with MongoDB, implementing persistent event processing, enforcing automation ownership and permissions, preventing duplicate execution, supporting scheduled follow-ups, and generating analytics from persistent application data.

The existing Automation and Analytics architecture and API contracts should be preserved wherever practical.

1.1 Phase II Objectives

The following responsibilities must be completed:

- Replace in-memory AutomationRule and ActivityEvent storage with MongoDB using Mongoose.
- Implement persistent automation rules and event history.
- Maintain relationships between AutomationRules, Users, Campaigns, and business entities.
- Protect automation and analytics endpoints using JWT authentication.
- Enforce role-based authorization and resource ownership.
- Implement event-based automation processing.
- Support relevant business triggers.
- Prevent duplicate event processing and duplicate automation execution.
- Implement scheduled follow-up actions.
- Integrate automation actions with the agreed Delivery service contract.
- Generate analytics summaries from MongoDB.
- Restrict analytics access according to user roles and authorized resources.
- Preserve existing Phase I API functionality.
- Implement Postman and automated tests for persistence, validation, event processing, and authorization.

The Automation and Analytics module must remain independently developable and testable without requiring completion of the other feature modules.

---

2. Development Environment and Team Independence

The Dockerized backend and MongoDB infrastructure will be prepared by Member 1 and merged into `main` before Phase II feature development begins.

Once the infrastructure is available, the repository must be cloned or updated from the latest `main` branch.

Development must occur in:

`feature/automation-p2-m5`

The shared Docker configuration, MongoDB connection, JWT authentication middleware, role constants, and error-handling contracts must be reused.

Member 5 must not independently replace or modify shared infrastructure.

The Automation and Analytics module must remain responsible for its own:

- Mongoose models.
- MongoDB repositories.
- Automation services and business rules.
- Controllers and routes.
- Event processing.
- Trigger matching.
- Scheduled automation execution.
- Duplicate event and execution prevention.
- Analytics queries and calculations.
- Module-specific authorization.
- Unit and integration tests.
- Module documentation.

Changes to shared files such as `backend/src/app.js`, `backend/src/routes/index.js`, `backend/src/config/`, or shared authentication middleware must be coordinated with Member 1.

2.1 Independent Development Requirements

The following approach must be followed:

1. Existing Automation and Analytics API contracts must be preserved wherever practical.
2. The in-memory repositories must be replaced without requiring changes to other members' modules.

3. Stable MongoDB identifiers must be used for User, Campaign, Lead, and event relationships.
4. Test fixtures may be used to create sample Users, AdvisorProfiles, Campaigns, AutomationRules, and ActivityEvents.
5. Shared authentication and relationship lookup contracts may be mocked during isolated tests.
6. The Automation module must not require the Leads, Campaigns, or Delivery modules to be completed before development begins.
7. Business events must use a documented event structure agreed upon by the team.
8. External actions must be implemented through adapter interfaces that can be mocked.
9. Automation processing must remain functional when delivery providers are unavailable.
10. Scheduled follow-ups must not require changes to the Delivery module's queue processing implementation.

The completed module must use the shared MongoDB connection and authentication middleware.

Other modules may publish business events through the agreed event contract after integration, but independent tests must be possible using the existing event simulation endpoint.

---

3. Shared User Roles and Permissions

The application supports three roles:

3.1 Administrator — `ADMIN`

Administrators manage platform-level user accounts, permissions, and administrative operations.

Administrative access to Automation and Analytics must follow explicitly defined permissions.

Administrators may review automation configuration and system activity only when authorized by the shared security policy.

The Administrator role must not automatically bypass all ownership checks without an intentional permission rule.

3.2 Financial Advisor — `ADVISOR`

Financial Advisors manage automation rules and marketing performance information associated with their own resources.

An authenticated Advisor must be allowed to:

- Create automation rules under their own account.
- View their own automation rules.
- Enable or disable their own rules.
- Configure supported triggers and actions.
- Associate permitted Campaigns with automation rules.
- Schedule authorized follow-up actions.
- View automation activity associated with their own resources.
- View analytics for their own Leads, Campaigns, and DeliveryJobs.

An Advisor must not access or modify another Advisor's private automation rules or analytics.

3.3 Marketing Specialist — `MARKETING_SPECIALIST`

Marketing Specialists may perform permitted automation and analytics activities for assigned Campaigns.

A Marketing Specialist may:

- View permitted automation activity related to assigned Campaigns.
- View authorized Campaign analytics.
- Review the results of permitted automated marketing actions.
- Manage explicitly permitted automation rules when authorized.

A Marketing Specialist must not:

- Access unrelated automation rules.
- View analytics belonging to unrelated Advisors.
- Modify automation rules owned by another user without authorization.
- Create unrestricted automation actions against arbitrary Leads or Campaigns.
- Bypass recipient consent requirements.
- Directly execute unauthorized Delivery operations.

- Access sensitive Lead contact information through analytics.
- Modify AdvisorProfile information.
- Manage advisor-specialist invitations.
- Assign themselves to Campaigns.
- Perform administrative operations.

A Marketing Specialist must satisfy both conditions before accessing Campaign-related automation resources:

1. An accepted advisor-specialist relationship exists.
2. The Marketing Specialist is assigned to the relevant Campaign.

For Phase II, new automation rule creation and rule ownership should remain with Financial Advisors by default.

Specialists may view permitted Campaign-related automation information and manage explicitly authorized rule settings.

The backend must enforce all permissions.

---

4. Database Identifiers and Relationships

The following identifier conventions must be used consistently.

4.1 Identifier Definitions

Field	Meaning	Referenced Collection
`userId`	Unique identifier of a registered user	`users`
`advisorId`	Unique identifier of an AdvisorProfile	`advisorProfiles`
`specialistUserId`	User ID belonging to a Marketing Specialist	`users`
`ownerId`	User ID of the owner of an AutomationRule or ActivityEvent	`users`

`automationRuleId`	Unique identifier of an AutomationRule	`automationRules`
`campaignId`	Unique identifier of a Campaign	`campaigns`
`leadId`	Unique identifier of a Lead	`leads`
`deliveryJobId`	Unique identifier of a DeliveryJob	`deliveryJobs`
`entityId`	Identifier of the entity referenced by an ActivityEvent	Determined by `entityType`
`eventId`	Unique application-level identifier used for event deduplication	`activityEvents`

Important distinctions:

`advisorId` always references `AdvisorProfile._id`, not `User._id`.

`ownerId` references `User._id`.

For example:

- `User._id = A`
- `AdvisorProfile._id = B`
- `AdvisorProfile.userId = A`
- `AutomationRule.ownerId = A`
- `Campaign.advisorId = B`

An AutomationRule belonging to an Advisor must use the Advisor's User ID as `ownerId`.

The associated Campaign must still be validated using the AdvisorProfile relationship.

For a Marketing Specialist:

- `User._id = C`
- `Campaign.assignedSpecialistIds = [C]`

Marketing Specialists do not have AdvisorProfile records.

Event identifier distinction

`ActivityEvent._id` is the MongoDB document identifier.

`ActivityEvent.eventId` is the unique event identifier used to prevent duplicate event processing.

These fields must not be treated as interchangeable.

4.2 Required Relationships

User → AutomationRule

Relationship: One-to-many.

One User may own multiple AutomationRules.

Each AutomationRule must reference a valid User through:

`AutomationRule.ownerId → User._id`

Campaign → AutomationRule

Relationship: One-to-many when a Campaign reference is present.

An AutomationRule may reference a Campaign through:

`AutomationRule.campaignId → Campaign._id`

The Campaign must belong to the Advisor associated with the rule owner, or the relationship must otherwise be explicitly authorized.

Rules that do not require a Campaign may have a null Campaign reference.

User → ActivityEvent

Relationship: One-to-many.

An ActivityEvent may reference the relevant owning User through:

`ActivityEvent.ownerId → User._id`

Event ownership must not be determined solely from untrusted client-submitted values in production event processing.

Business Entity → ActivityEvent

Relationship: One-to-many.

An ActivityEvent may reference different business entities.

For example:

- `entityType = "LEAD"` and `entityId = Lead._id`
- `entityType = "CAMPAIGN"` and `entityId = Campaign._id`
- `entityType = "DELIVERY"` and `entityId = DeliveryJob._id`

The meaning of `entityId` must be determined by `entityType`.

AutomationRule → ActivityEvent

An ActivityEvent may cause one or more AutomationRules to match.

An AutomationRule may process multiple events.

This is a logical many-to-many relationship.

Duplicate processing must be prevented using event identifiers and durable execution tracking.

The existing ActivityEvent collection may be extended with appropriate execution metadata, or a separate execution collection may be proposed if necessary.

Any additional collection must be documented and remain within Member 5's ownership.

AutomationRule → DeliveryJob

AutomationRules may request delivery actions through the agreed Delivery service contract.

The Automation module must not directly modify DeliveryJob status or invoke external email providers.

Delivery execution remains the responsibility of Member 4.

---

5. Member 5 — Assigned Phase II Responsibilities

5.1 AutomationRule Mongoose Model and Persistence

Owned directory: `backend/src/modules/automation/`

The existing AutomationRule model must be reviewed and updated for persistent MongoDB storage.

Collection:

`automationRules`

Recommended fields include:

- `_id`
- `ownerId`
- `name`
- `description`
- `trigger`
- `action`
- `campaignId`
- `delayMinutes`
- `enabled`
- `lastExecutedAt`
- `createdAt`
- `updatedAt`

Additional execution metadata may be introduced where necessary.

The existing Phase I model fields and enum values must be preserved wherever practical.

Requirements include:

1. Required field validation.
2. Correct Mongoose field types.

3. Supported trigger values.
4. Supported action values.
5. Appropriate default values.
6. Timestamp management.
7. User ownership references.
8. Optional Campaign references.
9. Appropriate database indexes.

The `ownerId` field must reference `User._id`.

A rule must not be allowed to reference another Advisor's private Campaign without the appropriate authorization.

5.2 ActivityEvent Mongoose Model and Persistence

Collection:

`activityEvents`

Recommended fields include:

- `_id`
- `eventId`
- `eventType`
- `entityType`
- `entityId`
- `ownerId`
- `metadata`
- `occurredAt`
- `createdAt`
- `updatedAt`

Additional processing metadata may include:

- `processingStatus`
- `processedAt`
- `processingError`

The model must support relevant business events from Leads, Campaigns, Delivery, and Automation.

Requirements include:

- Unique event identifiers.
- Validation of supported event types.
- Validation of supported entity types.
- Valid MongoDB references.
- Appropriate timestamps.
- Safe metadata storage.
- Relevant indexes for event retrieval and analytics.
- Duplicate event prevention.

A unique index must be created for `eventId`.

Event metadata must not contain unnecessary sensitive user information, credentials, passwords, or JWTs.

5.3 Replace In-Memory Repositories

Primary file: `backend/src/modules/automation/automationRepository.js`

The current repository stores AutomationRules and ActivityEvents in temporary JavaScript arrays.

These operations must be replaced with asynchronous Mongoose queries.

Required repository functionality includes:

- Create AutomationRule.
- Retrieve AutomationRule by ID.
- List AutomationRules.
- Filter AutomationRules.
- Update rule execution metadata.
- Create ActivityEvent.
- Retrieve ActivityEvent by eventId.
- List ActivityEvents.
- Filter ActivityEvents.
- Prevent duplicate event creation.
- Retrieve enabled rules matching an event.
- Retrieve scheduled actions awaiting execution.
- Calculate event and rule counts.

Database queries must remain in the repository layer.

Controllers must not directly query MongoDB.

Automation services must enforce business rules.

Records must remain available after backend restart.

MongoDB filtering and aggregation should be used where appropriate instead of loading entire collections into application memory.

5.4 Automation Rule Creation

Existing endpoint:

```
POST /api/v1/automations
```

Automation rule creation must require JWT authentication.

For Phase II, only an authenticated Advisor or explicitly authorized Administrator should be allowed to create an AutomationRule.

The creation workflow must:

1. Authenticate the requesting user.
2. Verify the permitted role.
3. Determine the authenticated User ID.
4. Set `ownerId` using the authenticated identity.
5. Validate the rule name and description.
6. Validate the trigger.
7. Validate the action.
8. Validate any Campaign reference.
9. Validate the requested delay.
10. Apply appropriate defaults.
11. Save the rule to MongoDB.
12. Return the created rule.

The backend must not trust a client-submitted `ownerId` as proof of rule ownership.

If `ownerId` is included for API compatibility, it must match the authenticated User ID or be rejected.

Campaign validation

If the rule references a Campaign:

- The Campaign must exist.
- The Campaign must belong to the appropriate Advisor.
- The rule owner must be authorized to use the Campaign.
- The Campaign reference must use `Campaign._id`.

Member 5 must use the Campaign lookup and authorization contract rather than implementing Campaign ownership logic independently.

5.5 Automation Rule Retrieval and Management

Existing endpoint:

```
GET /api/v1/automations
```

Automation rules must be retrieved from MongoDB.

The endpoint must require authentication.

The following restrictions must be enforced:

Advisor access

Advisors may view rules that they own.

They must not retrieve another Advisor's private automation rules.

Marketing Specialist access

Specialists may view permitted Campaign-related automation information when both the accepted advisor relationship and Campaign assignment requirements are satisfied.

Specialists must not receive unrestricted access to another user's automation configuration.

Administrator access

Administrative access must follow explicitly defined permissions.

Rule management

AutomationRules must support controlled activation and deactivation.

Recommended additional endpoint:

`PATCH /api/v1/automations/:id`

Permitted updates may include:

- Name.
- Description.
- Trigger.
- Action.
- Campaign reference.
- Delay.
- Enabled status.

The `ownerId`, creation timestamp, and execution metadata must not be freely modifiable through ordinary update requests.

Rule updates must verify ownership and authorization.

5.6 Event Processing and Trigger Matching

The existing Phase I simulation functionality must be extended to support real persistent event processing.

Existing endpoint:

`POST /api/v1/automations/simulate-event`

This endpoint must remain available for testing, but must be protected in Phase II.

The completed event-processing service must support events generated by authorized application operations.

Shared event contract

Recommended structure:

```
None
{
  "eventId": "evt_example_001",
  "eventType": "LEAD_CREATED",
```

```
  "entityType": "LEAD",
  "entityId": "665f1c2a9b1e8a0012a4c101",
  "ownerId": "665f1c2a9b1e8a0012a4c001",
  "metadata": {}
}
```

In this example, `ownerId` must represent the relevant User ID, not the AdvisorProfile ID.

The existing supported event types must be preserved.

Additional event types may be introduced through the shared event contract.

Recommended events include:

- `LEAD_CREATED`
- `CONSULTATION_REQUESTED`
- `CAMPAIGN_CREATED`
- `CAMPAIGN_ACTIVATED`
- `CAMPAIGN_COMPLETED`
- `DELIVERY_QUEUED`
- `DELIVERY_SENT`
- `DELIVERY_FAILED`

Only event types included in the agreed model and contract must be accepted.

Processing workflow

When an event is received:

1. Validate the event structure.
2. Verify the event's source and ownership.
3. Check whether the eventId has already been processed.
4. Persist the event.
5. Retrieve enabled AutomationRules matching the event type.
6. Verify each rule's conditions and ownership.
7. Determine the required action.
8. Execute or schedule the action through the appropriate adapter.
9. Record the result.

10. Update execution metadata.

A simulated event must be clearly distinguished from a production event.

The simulation endpoint must not become a public mechanism for triggering unauthorized marketing actions.

For example, an untrusted client must not be able to submit another Advisor's `ownerId` to trigger their automation rules.

Production events must originate from trusted backend operations or a validated internal event contract.

5.7 Duplicate Event and Execution Prevention

The current Phase I implementation performs duplicate event checks using `eventId`.

Phase II must enforce duplicate prevention through MongoDB.

Event deduplication

A unique MongoDB index on `ActivityEvent.eventId` must prevent duplicate event records.

If an event with the same `eventId` is submitted again, it must not be processed as a new event.

The API must return a consistent result according to the documented duplicate handling policy.

Automation execution deduplication

Preventing duplicate ActivityEvents alone is not sufficient.

The same event may match several AutomationRules, and each rule must execute no more than once for that event.

A durable execution identity must therefore be maintained using:

```
eventId + automationRuleId
```

This combination should be unique.

The system must prevent duplicate actions when:

- An event is submitted more than once.
- Two processing requests occur concurrently.
- The backend restarts during processing.
- A scheduled action is retrieved by multiple processing attempts.

Atomic MongoDB operations and appropriate unique indexes must be used.

Failure handling

A failed action must be recorded.

Retries, if implemented, must follow a documented policy.

An action whose external outcome is uncertain must not be blindly repeated.

For delivery-related actions, duplicate prevention must also use the Delivery module's idempotency and queueing contract.

5.8 Scheduled Follow-Up Actions

AutomationRules must support delayed actions.

The existing `delayMinutes` field must be used.

For example:

- Trigger: `CONSULTATION_REQUESTED`
- Action: `SCHEDULE_FOLLOW_UP`
- Delay: 1440 minutes

The required scheduled execution time must be calculated from the event occurrence time and configured delay.

Scheduling requirements

The implementation must:

1. Validate the configured delay.
2. Determine the scheduled execution time.
3. Persist the scheduled action.
4. Retrieve actions that are due.

5. Prevent premature execution.
6. Prevent duplicate execution.
7. Record completion or failure.
8. Preserve scheduling information after backend restart.

A scheduled action must not exist only in a JavaScript timer or process memory.

A persistent scheduling mechanism must be used.

A lightweight worker or polling mechanism may be implemented within the Automation module using the shared MongoDB connection.

The implementation must remain compatible with the Dockerized backend.

No additional infrastructure service should be required unless explicitly coordinated with Member 1.

Responsibility boundary

The Automation module decides **when an automation action should be triggered**.

The Delivery module decides **whether a delivery is eligible and how it is queued or processed**.

For example, when an automation schedules a follow-up email, it must use the agreed Delivery service contract.

The Automation module must not directly send email through Gmail or publish Facebook content.

5.9 Automation Actions and Delivery Integration

Supported actions must follow the existing AutomationRule model.

Examples include:

- `SEND_EMAIL`
- `SCHEDULE_FOLLOW_UP`

Additional actions may be implemented only when supported by the agreed event and action contracts.

SEND_EMAIL

When an eligible event matches an enabled rule:

1. Confirm the rule is authorized for the event owner.
2. Validate the associated Campaign reference.
3. Determine the required delivery request.
4. Pass the request through the agreed Delivery service interface.
5. Record the action outcome.

The Delivery module remains responsible for:

- Campaign status validation.
- Recipient eligibility.
- Marketing consent.
- Unsubscribe checks.
- Provider selection.
- Delivery queue persistence.
- Rate limiting.
- Email sending.
- Facebook publishing.
- Delivery status tracking.

SCHEDULE_FOLLOW_UP

When an event matches a follow-up rule:

1. Calculate the execution time.
2. Persist the scheduled action.
3. Prevent duplicate scheduling.
4. Execute the action when due.
5. Record the result.

A follow-up may ultimately create a permitted DeliveryJob, but it must not bypass Delivery validation.

Independent development

During isolated testing, a mock Delivery adapter must be used.

The mock adapter must return predictable success or failure responses.

The Automation module must not require Gmail credentials, Facebook permissions, or a running Delivery worker for its own tests.

5.10 Activity Event History

Existing endpoint:

`GET /api/v1/automations/events`

Event history must be retrieved from MongoDB.

The endpoint must require authentication.

Supported filters should include:

- Event type.
- Owner.
- Entity.
- Date range.
- Processing status where applicable.

The implementation must:

- Enforce event ownership.
- Restrict access by role.
- Support appropriate filtering.
- Return stable sorting.
- Use database-backed pagination where appropriate.
- Avoid exposing sensitive event metadata.
- Preserve processing outcomes.
- Return structured errors for invalid queries.

Advisors may view events associated with their own resources.

Marketing Specialists may view only permitted Campaign-related activity.

An accepted advisor-specialist relationship alone must not authorize access to all of an Advisor's events.

5.11 Analytics Using MongoDB

Existing endpoint:

`GET /api/v1/analytics/summary`

The existing Phase I analytics endpoint currently summarizes in-memory AutomationRules, ActivityEvents, and simulated actions.

Phase II must replace these calculations with MongoDB-backed queries.

Required automation metrics

The analytics summary must retain existing functionality such as:

- Total AutomationRules.
- Enabled rules.
- Disabled rules.
- Rules grouped by trigger.
- Rules grouped by action.
- Total ActivityEvents.
- Events grouped by type.
- Events with matching rules.
- Simulated or executed action counts, clearly distinguished.

Recommended business metrics

Where the agreed data contracts make them available, the analytics module should also support:

- Total Leads.
- Leads grouped by status.
- Active Campaign count.
- Completed Campaign count.
- Consultation request count.
- Converted Lead count.
- DeliveryJobs grouped by status.
- Successful delivery count.
- Failed delivery count.

Lead conversion rate

A consistent conversion-rate definition must be documented.

Recommended calculation:

```
Conversion Rate = (Converted Leads / Total Leads) × 100
```

When Total Leads equals zero, the conversion rate must be reported as zero or according to another clearly documented convention.

Database aggregation

MongoDB aggregation pipelines and count queries should be used where appropriate.

Analytics should not retrieve entire collections into application memory solely to calculate counts.

Analytics access restrictions

An Advisor must see analytics associated with their own records.

A Marketing Specialist must see only metrics related to Campaigns they are authorized to access.

Cross-advisor totals must not be returned to ordinary Advisors or Marketing Specialists unless explicitly authorized.

For example, a specialist assigned to one Campaign must not automatically receive all Leads or revenue-related information belonging to the Advisor.

The analytics service must enforce these restrictions on the backend.

5.12 Secure Data Handling

The Automation and Analytics module must implement:

- JWT authentication.
- Role-based authorization.
- Automation rule ownership checks.
- Campaign assignment verification where applicable.
- Validation of MongoDB ObjectIds.
- Validation of supported triggers and actions.
- Validation of event types.
- Validation of event payloads.
- Protection against mass assignment.
- Duplicate event prevention.
- Duplicate execution prevention.
- Safe event metadata handling.
- Secure scheduled-action processing.

- Controlled error responses.
- Appropriate HTTP status codes.
- Restrictions on unauthorized event simulation.
- Protection against cross-advisor analytics exposure.

The shared authentication middleware must be reused.

Member 5 must not implement a separate JWT authentication system.

Fields such as `ownerId`, `lastExecutedAt`, `processingStatus`, and internal execution metadata must not be freely modifiable through ordinary client requests.

Automation execution must occur through controlled backend operations.

---

6. Internal Contracts for Other Modules

The Automation and Analytics module must support integration with Identity, Leads, Campaigns, and Delivery.

These integrations must be mockable during independent development.

6.1 Identity Integration

Member 1 provides:

- JWT authentication.
- Role validation.
- User lookup.
- AdvisorProfile lookup.
- Accepted advisor-specialist relationship lookup.

Member 5 must use these contracts to verify automation ownership and authorized analytics access.

Member 5 must not implement or modify the invitation system.

6.2 Leads Integration

Member 2 provides:

- Lead existence and ownership information.
- Lead status.
- Consultation request information.
- Lead conversion information.
- Marketing consent information where required.

Lead operations may publish events such as:

- `LEAD_CREATED`
- `LEAD_STATUS_CHANGED`
- `CONSULTATION_REQUESTED`

The Automation module must define and document the event contract used to process these events.

Member 5 must not modify CRM lifecycle logic or implement Lead storage.

6.3 Campaigns Integration

Member 3 provides:

- Campaign existence lookup.
- Campaign ownership.
- Campaign status.
- Campaign-specific specialist assignments.
- Authorized Campaign access checks.

Campaign operations may publish events such as:

- `CAMPAIGN_CREATED`
- `CAMPAIGN_ACTIVATED`
- `CAMPAIGN_COMPLETED`

Member 5 must use the Campaign service contract rather than independently implementing Campaign ownership rules.

The Automation module must not modify Campaign status or specialist assignments.

6.4 Delivery Integration

Member 4 provides:

- Delivery queueing.
- Recipient validation.
- Consent enforcement.
- Delivery scheduling.
- Provider integration.
- Rate limiting.
- Delivery status tracking.

Automation may request authorized delivery actions through the agreed service interface.

Delivery operations may publish events such as:

- `DELIVERY_QUEUED`
- `DELIVERY_SENT`
- `DELIVERY_FAILED`

Member 5 must not implement email sending or Facebook publishing.

Event-loop prevention

Automation rules must not create uncontrolled processing loops.

For example, a `DELIVERY_QUEUED` event must not repeatedly create another identical DeliveryJob.

Event correlation, deduplication, and permitted trigger-action combinations must be used to prevent recursive execution.

---

7. API Compatibility

The following Phase I endpoints must remain functional:

Method	Endpoint	Purpose
POST	`/api/v1/automations`	Create AutomationRule
GET	`/api/v1/automations`	List AutomationRules

POST	`/api/v1/automations/simulate-event`	Simulate business event
GET	`/api/v1/automations/events`	Retrieve ActivityEvents
GET	`/api/v1/analytics/summary`	Retrieve analytics summary

These endpoints must be updated to use persistent MongoDB storage and shared authentication.

The existing route paths should remain unchanged.

Recommended additional endpoint:

Method	Endpoint	Purpose
PATCH	`/api/v1/automations/:id`	Update or enable/disable a rule

The existing response formats should be preserved wherever practical.

Any necessary changes must be documented.

Simulation endpoint restriction

The simulation endpoint is intended for controlled testing and demonstrations.

It must not be exposed as an unrestricted production action endpoint.

Access must be protected according to the shared security policy.

A simulated event must not send real marketing communications unless a separate, explicitly authorized test operation is implemented.

---

8. Testing Requirements

The Automation and Analytics module must be tested using Postman and appropriate automated tests.

Tests must use MongoDB persistence and controlled sample data.

External delivery providers must be mocked during automated tests.

At minimum, the following scenarios must be verified.

8.1 Database Persistence

- AutomationRule creation stores records in MongoDB.
- ActivityEvent creation stores records in MongoDB.
- Rules can be retrieved after creation.
- Events can be retrieved after creation.
- Rule updates persist.
- Records remain available after backend restart.
- Invalid database records are rejected.
- Required indexes and schema validation are applied.

8.2 Authentication and Authorization

- Missing JWT is rejected.
- Invalid JWT is rejected.
- Expired JWT is rejected.
- Advisors can create their own rules.
- Advisors cannot modify another Advisor's rules.
- Client-supplied `ownerId` cannot override authenticated ownership.
- Unassigned specialists cannot access private Campaign automation.
- Specialists with accepted relationships and Campaign assignments can view permitted information.
- Revoked relationships invalidate related specialist access.
- Unauthorized users cannot access private analytics.

8.3 Automation Rule Validation

- Valid rule creation succeeds.
- Missing required fields are rejected.
- Invalid trigger values are rejected.
- Invalid action values are rejected.

- Invalid delay values are rejected.
- Invalid Campaign references are rejected.
- Unauthorized Campaign references are rejected.
- Disabled rules are not executed.
- Rule updates enforce ownership.

8.4 Event Processing

- Valid events are persisted.
- Matching enabled rules are identified.
- Unmatched events are handled correctly.
- Disabled rules are ignored.
- Invalid event types are rejected.
- Invalid entity references are rejected.
- Event ownership is verified.
- Unauthorized event simulation is rejected.
- Simulated actions do not invoke real providers unintentionally.

8.5 Duplicate Prevention

- Duplicate eventIds are rejected or handled idempotently according to the documented contract.
- Concurrent duplicate event submissions do not create duplicate records.
- The same rule does not execute twice for the same event.
- Separate rules may each execute once for the same event.
- Execution deduplication remains effective after restart.
- Failed actions follow the documented retry policy.
- Duplicate delivery requests are prevented through the agreed Delivery contract.

8.6 Scheduling

- Valid delayed actions are scheduled.
- Scheduled actions persist in MongoDB.
- Future actions do not execute early.
- Due actions execute through the appropriate adapter.
- Concurrent workers do not execute the same action twice.
- Scheduling remains valid after restart.
- Failed scheduled actions are recorded.
- Invalid delays are rejected.

8.7 Analytics

- Rule counts are accurate.
- Enabled and disabled rule counts are accurate.
- Trigger and action grouping is correct.
- Event counts are accurate.
- Event type grouping is correct.
- Lead counts are accurate where integrated.
- Campaign counts are accurate where integrated.
- Delivery result counts are accurate where integrated.
- Conversion rates use the documented formula.
- Empty datasets are handled correctly.
- Advisors see only authorized analytics.
- Specialists see only permitted Campaign-related analytics.
- MongoDB queries and aggregations return consistent results.

8.8 Integration Contract Tests

- AutomationRules reference valid Users.
- Campaign references are validated.
- Event metadata follows the agreed shared contract.
- Lead events can be processed through a mock event source.
- Campaign events can be processed through a mock event source.
- Delivery events can be processed through a mock event source.
- Automation actions can use a mock Delivery adapter.
- Duplicate event processing does not create duplicate Delivery requests.
- Automation remains functional when other feature modules are mocked.

---

9. Documentation and GitHub Requirements

Member 5 must provide module-specific documentation explaining:

- Automation and Analytics responsibilities.
- MongoDB AutomationRule schema.
- MongoDB ActivityEvent schema.

- Field definitions and validation rules.
- User ownership and Campaign relationships.
- Identifier conventions.
- Supported triggers.
- Supported actions.
- Event structure and event types.
- Trigger matching and processing.
- Duplicate event prevention.
- Duplicate rule execution prevention.
- Scheduled follow-up processing.
- Retry and failure-handling behavior.
- Delivery integration contract.
- Analytics metrics and calculations.
- Role-based authorization.
- Specialist access restrictions.
- API endpoint usage.
- Postman test scenarios.
- Any necessary changes from Phase I.

The documentation must explicitly state:

`advisorId` references `AdvisorProfile._id`, not `User._id`.

`ownerId` references `User._id`.

`campaignId` references `Campaign._id`.

`entityId` references the collection determined by `entityType`.

`eventId` is the unique event identifier used for deduplication and is different from `ActivityEvent._id`.

Marketing Specialist access to Campaign-related automation requires an accepted advisor relationship and a valid Campaign assignment.

No database credentials, JWT secrets, or sensitive event information must be committed to GitHub.

Development must occur in:

`feature/automation-p2-m5`

Changes must be committed with meaningful messages.

A pull request must be submitted targeting `main`.

The pull request must include:

- Completed functionality summary.
- Schema and repository changes.
- Authentication and authorization integration.
- Event processing implementation.
- Scheduling implementation.
- Analytics implementation.
- Tests performed.
- Relevant setup instructions.
- Known limitations, if applicable.

The module must remain independently testable before merging.

---

10. Completion Criteria

Member 5's Phase II responsibilities will be considered complete when:

- AutomationRule and ActivityEvent models are integrated with MongoDB using Mongoose.
- In-memory Automation and ActivityEvent storage has been replaced.
- Automation rules and event history remain available after restart.
- Required fields, defaults, indexes, and schema validation are implemented.
- AutomationRules reference valid Users.
- Optional Campaign references are validated.
- `ownerId` consistently means `User._id`.
- `advisorId` consistently means `AdvisorProfile._id` wherever used.
- Existing Automation and Analytics endpoints remain functional.
- Automation endpoints require JWT authentication.
- Advisor ownership is enforced by the backend.
- Marketing Specialists can access only permitted Campaign-related automation resources.
- Revoked advisor-specialist relationships invalidate related access.
- Automation rules support valid triggers and actions.

- Automation rules can be enabled and disabled through authorized operations.
- Business events are persisted and processed through the agreed event contract.
- Duplicate eventIds cannot cause repeated event processing.
- The same AutomationRule cannot execute twice for one event.
- Concurrent processing is controlled using persistent execution tracking.
- Scheduled follow-up actions are stored persistently.
- Scheduled actions execute only when due.
- Scheduling remains functional after backend restart.
- Automation actions use the agreed Delivery service contract.
- The Automation module does not directly send emails or publish Facebook content.
- Simulated events remain available for safe testing.
- Analytics summaries use MongoDB data.
- Analytics calculations are documented and accurate.
- Advisor-specific analytics are restricted by ownership.
- Specialist analytics are restricted to authorized resources.
- Other module event contracts are documented.
- No unrelated module's business logic has been implemented unnecessarily.
- Postman and automated tests cover successful and unsuccessful operations.
- Module documentation is complete.
- Changes are committed and submitted through a pull request.

Expected outcome: The Automation and Analytics module will operate using persistent MongoDB storage, with secure rule ownership, validated event processing, duplicate execution prevention, scheduled follow-ups, and database-backed analytics. Financial Advisors will manage automation associated with their own resources, while Marketing Specialists will access only explicitly permitted Campaign-related automation information. The module will integrate with Leads, Campaigns, and Delivery through stable backend contracts without depending on those modules for independent development and testing.